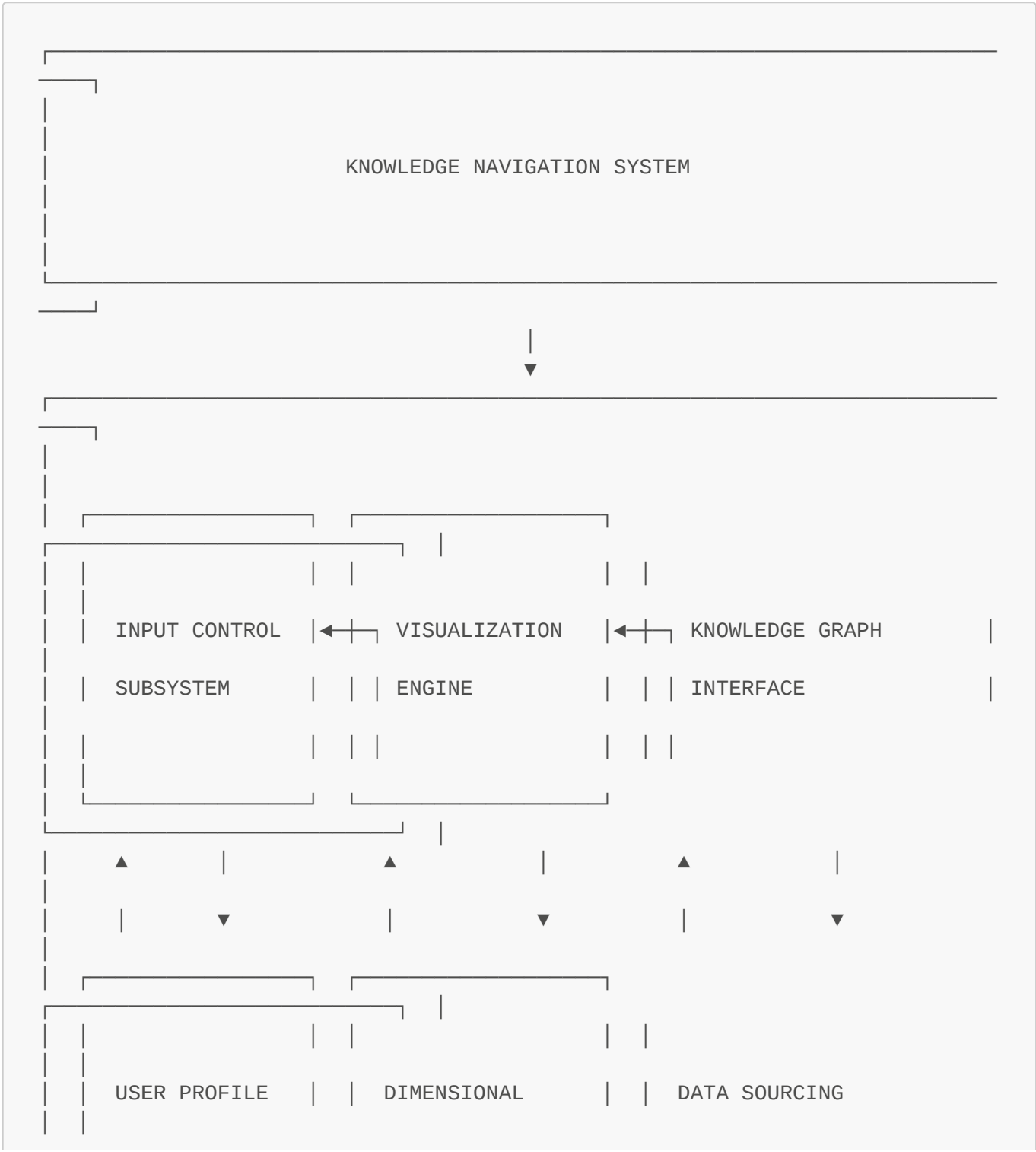
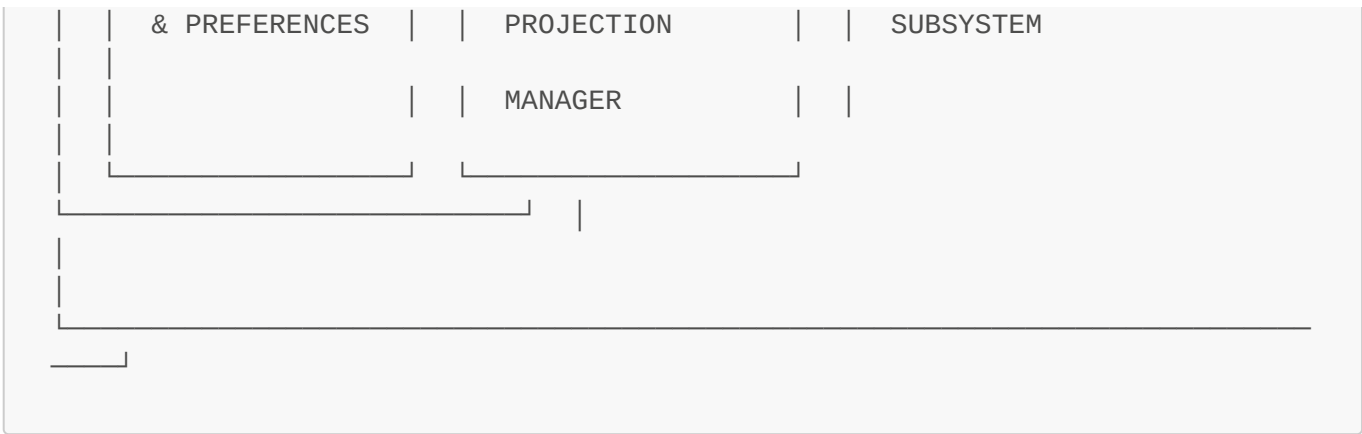
Knowledge Navigation Architecture

Created: April 7, 2025

System Overview

The Knowledge Navigation System provides an accessible interface for exploring multidimensional knowledge spaces through standard input devices. This document outlines the architectural components and their interactions.

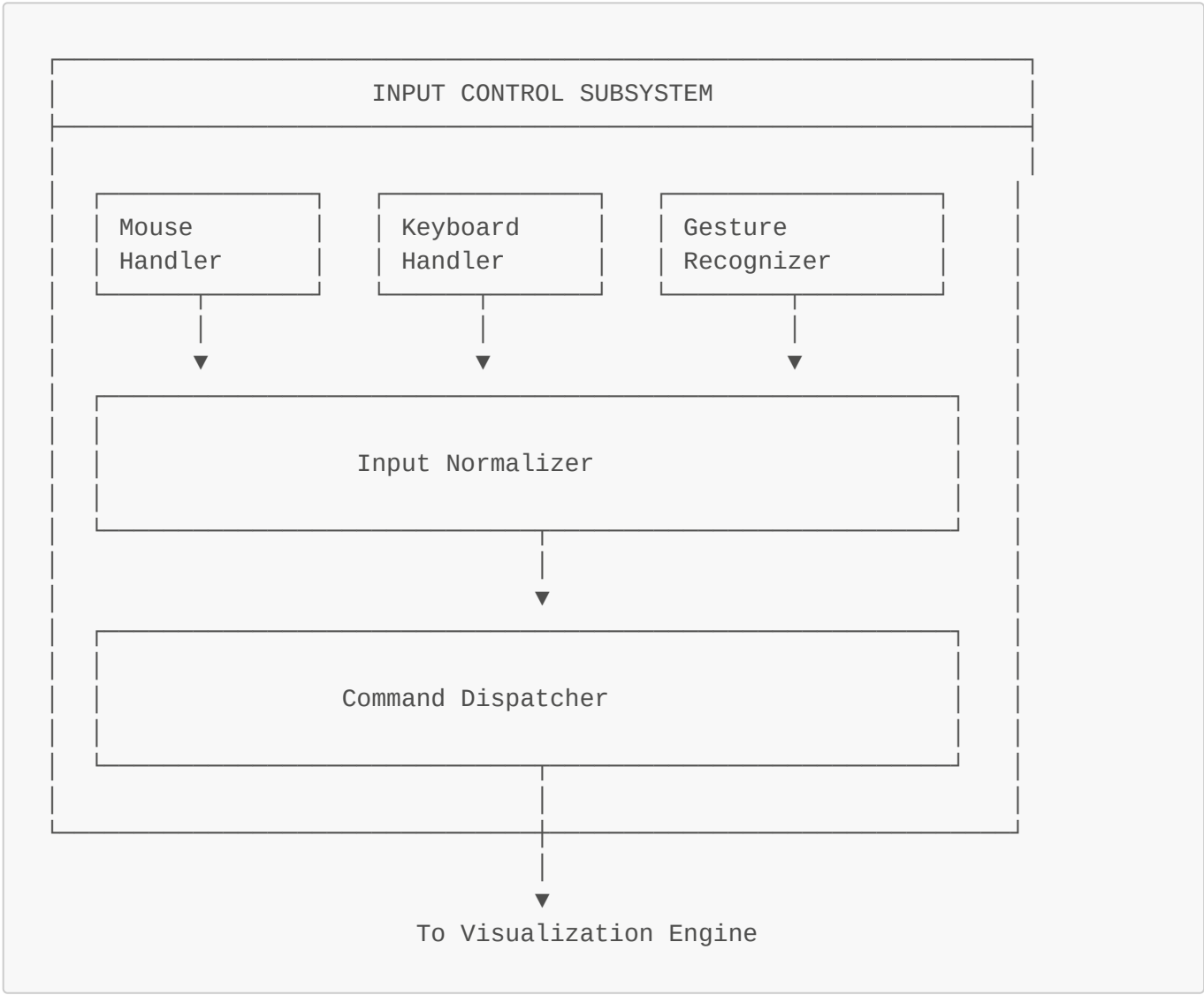




Core Components

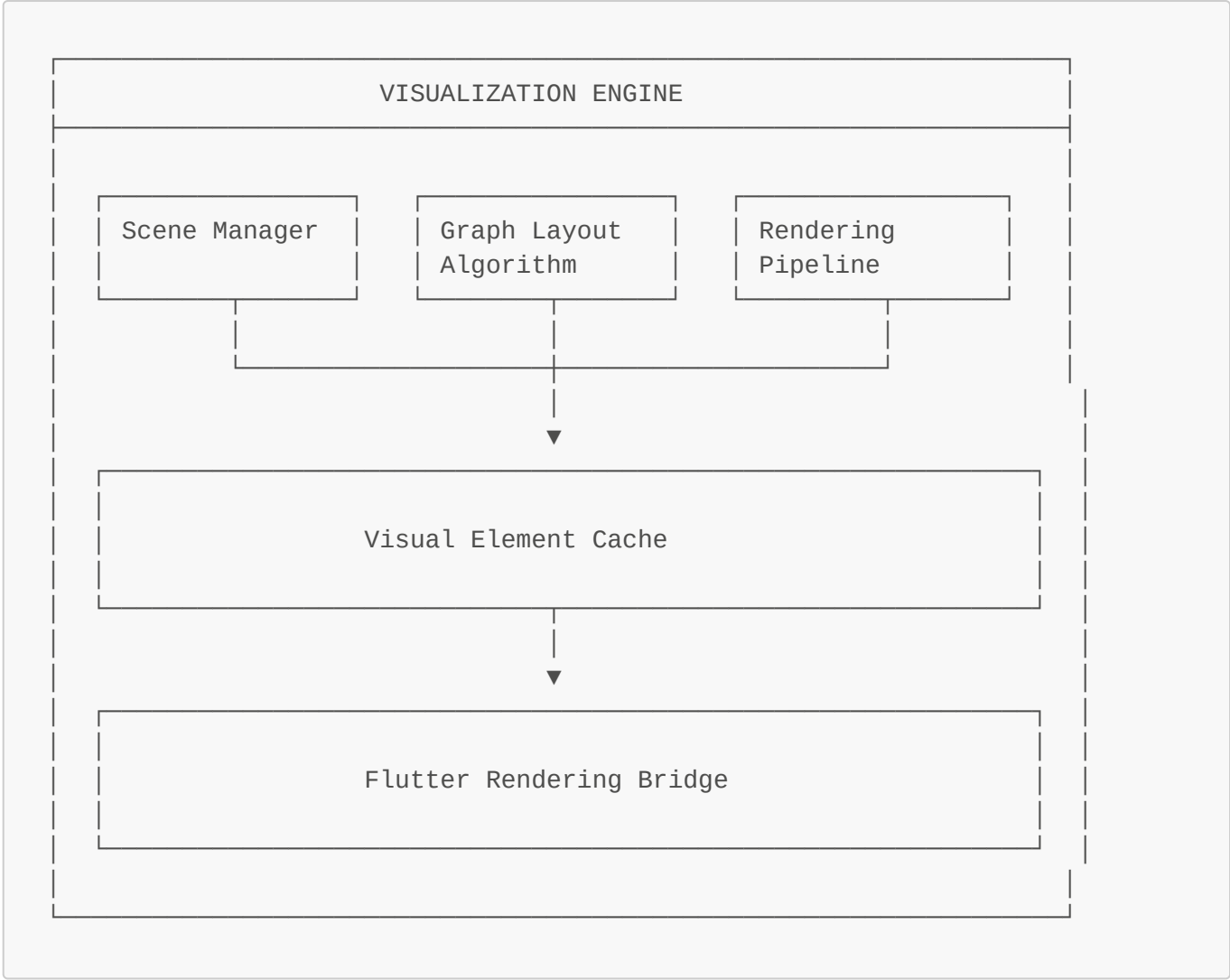
1. Input Control Subsystem

The input control subsystem manages user interactions through standard input devices, translating them into navigation commands within the knowledge space.



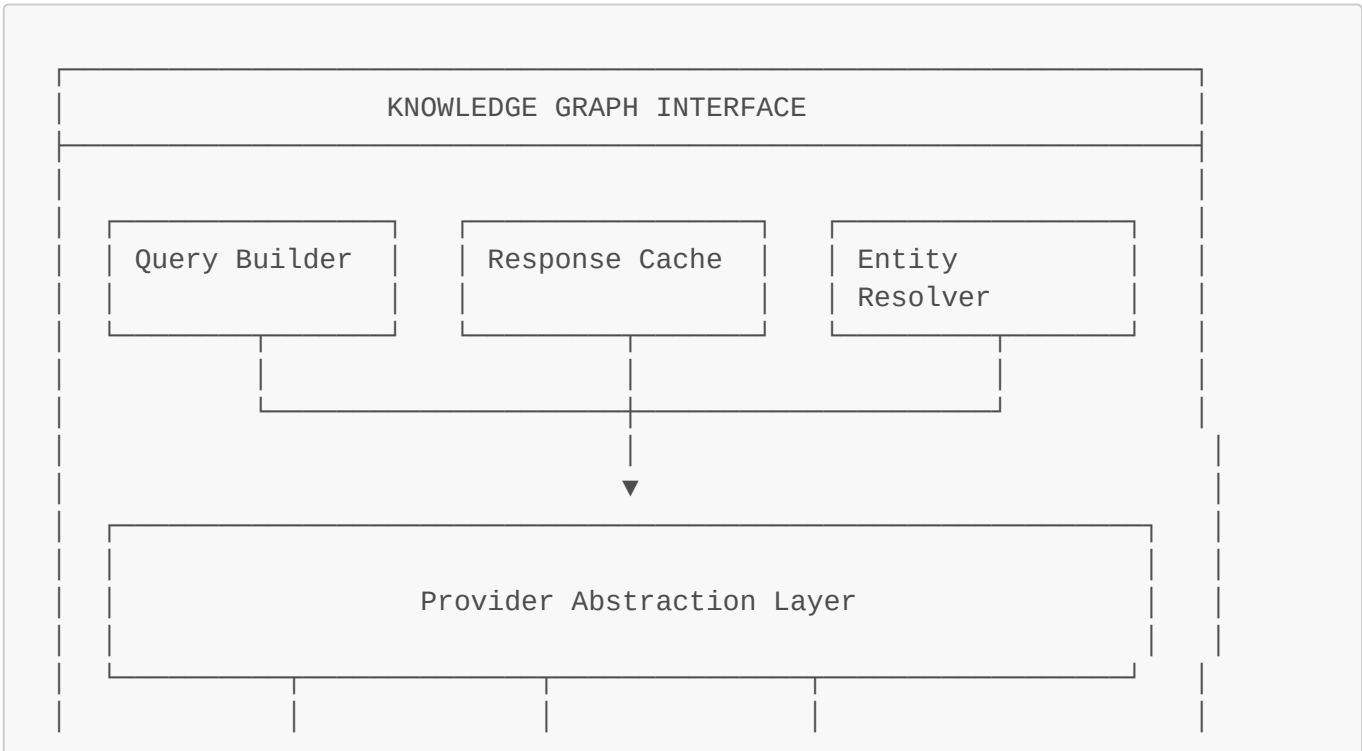
2. Visualization Engine

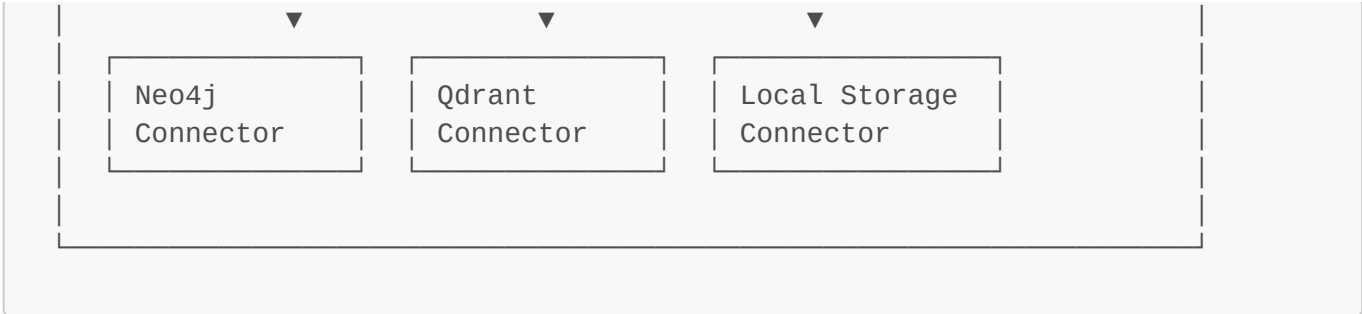
The visualization engine renders the knowledge graph and manages the visual representation of the multidimensional data space.



3. Knowledge Graph Interface

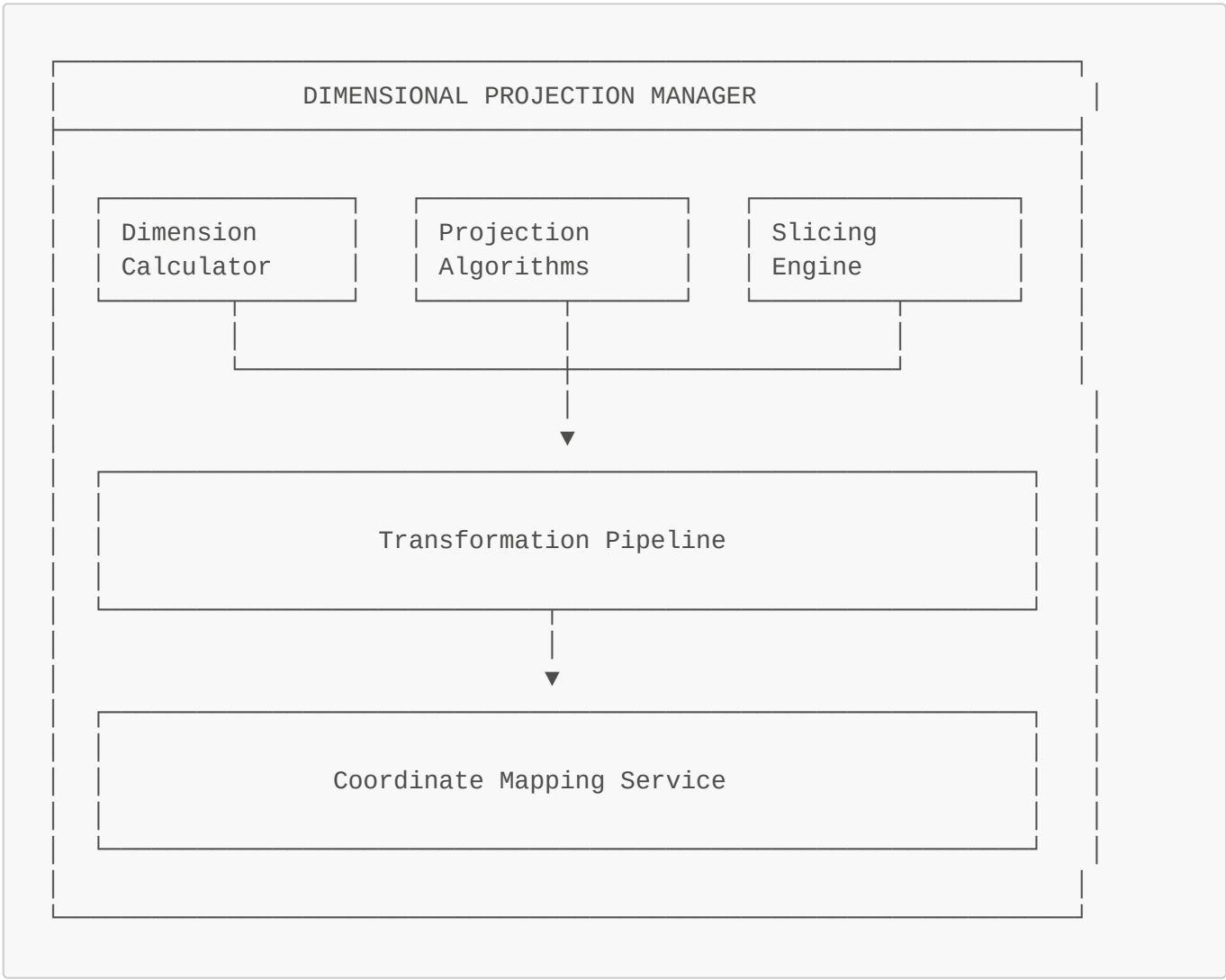
The knowledge graph interface connects to the backend storage systems and retrieves the relevant data for visualization.





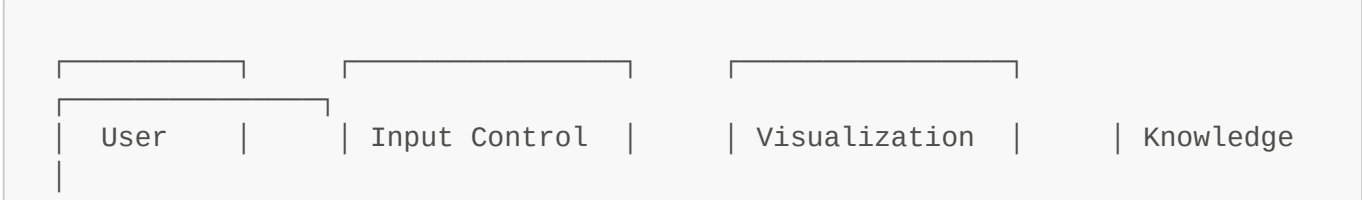
4. Dimensional Projection Manager

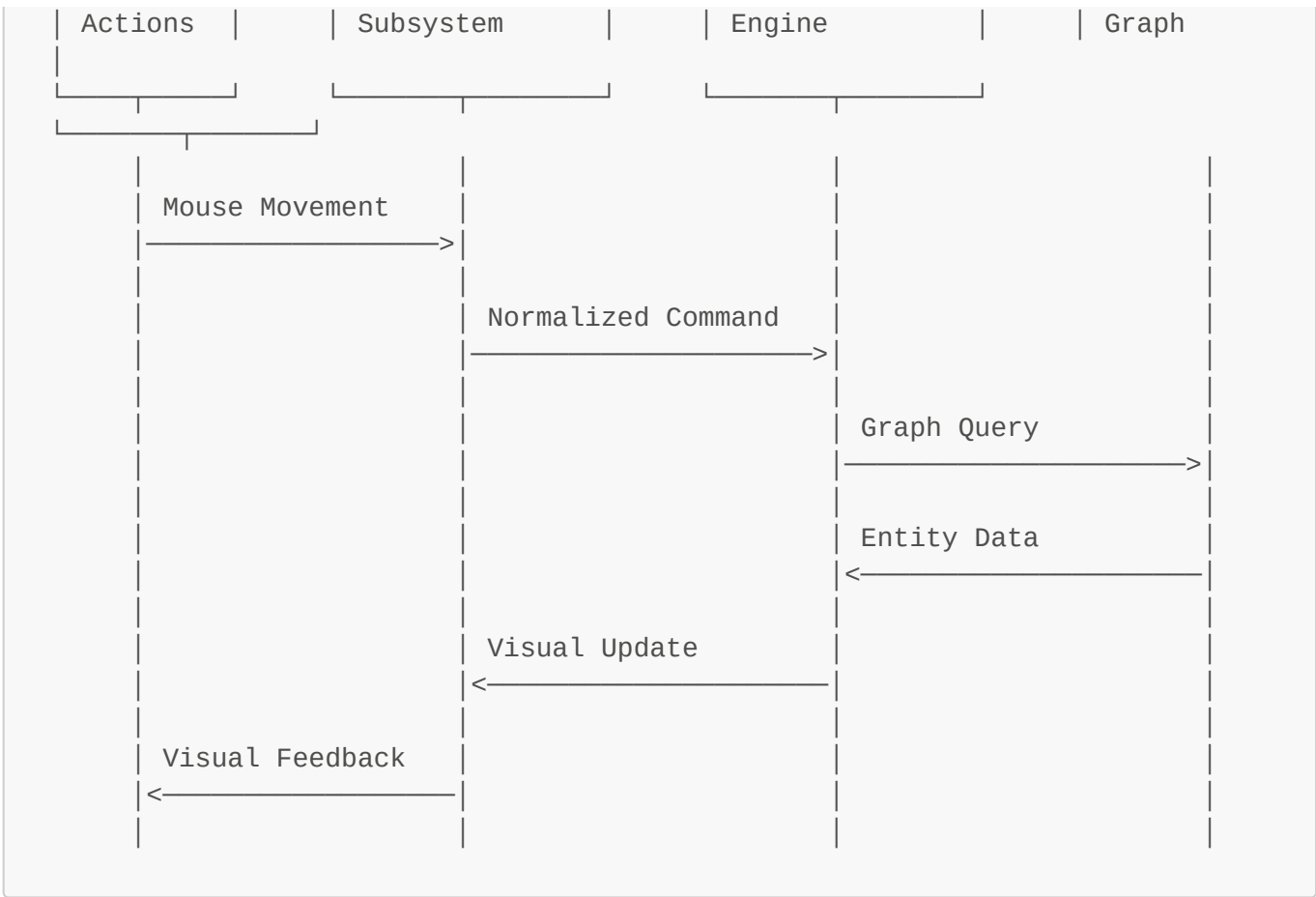
The dimensional projection manager handles the mathematical transformations necessary to represent high-dimensional data in a navigable 2D/3D space.



System Interactions

The following diagram illustrates the interactions between components during a typical knowledge exploration session:





User Control Mappings

Mouse Controls

- **Left-click + Drag**: Rotate the knowledge graph
- **Right-click**: Context menu for node/edge
- **Scroll Wheel**: Zoom in/out
- **Shift + Drag**: Pan across visualization
- **Ctrl + Click**: Select node/Add to selection

Keyboard Controls

- **Arrow Keys**: Navigate in cardinal directions
- **Page Up/Down**: Navigate dimensional elevation
- **1-9 Keys**: Toggle dimension visibility
- **Space**: Expand/collapse selected node
- **Tab**: Cycle between connected nodes
- **F1-F12**: Access saved views
- **+/-**: Adjust connection strength visibility
- **R**: Reset view to default

Implementation Considerations

1. **Performance Optimization**
    - Progressive loading of graph elements

    - Level-of-detail rendering
    - Cached projections for common views

2. **Accessibility**
    - Color schemes for colorblind users
    - Alternative navigation for mobility-impaired users
    - Keyboard equivalents for all mouse operations

3. **Extensibility**
    - Plugin system for custom visualizations
    - User-defined dimension mappings
    - Support for future input devices

Technology Stack

- **Frontend**: Flutter with custom painting and WebGL integration
- **Backend**: Neo4j for graph storage, Qdrant for vector operations
- **Algorithm Libraries**: TensorFlow.js for dimensionality reduction
- **Rendering**: Custom Dart implementation with shader support

---

This architecture aligns with Robinpedia's core principles of knowledge integration while providing an accessible interface for multidimensional knowledge exploration.